Data and Applications

Project Phase - 3

Team - Bitter Coleslaw

Leon Gilu

Sanjith Ganapathi

Sreevijay Sunil

Changes Made:

- We changed client_id attribute in the Case entity type from a primary key to foreign key referencing the client
- We added a foreign key called lawyer_assigned to replace the relationship lawyer (represents) client.
- We have changed the cardinality constraint of people_involved relation. It was earlier 1:n but we have corrected it to m:n as an operation can have multiple people involved.

Procedure to Convert ER diagram to Relational Model:

Strong Entities: Create a relation (table) for each strong entity. Include all the entity's simple attributes, using only the simple components of any composite attributes. Choose one of the entity's key attributes (or attribute set) as the primary key for the relation.

Multivalued Attribute: Create a new relation for the multivalued attribute. This relation must include the attribute itself (or its simple components in case of composite attribute) and the primary key of the entity it belongs to as a foreign key. The primary key of this new relation is the combination of the attribute and that foreign key.

Weak Entities: Create a relation for each weak entity. Add the primary key of the *owner* entity to this new relation as a foreign key. The primary key of the new relation is the combination of the owner's primary key and the weak entity's partial key.

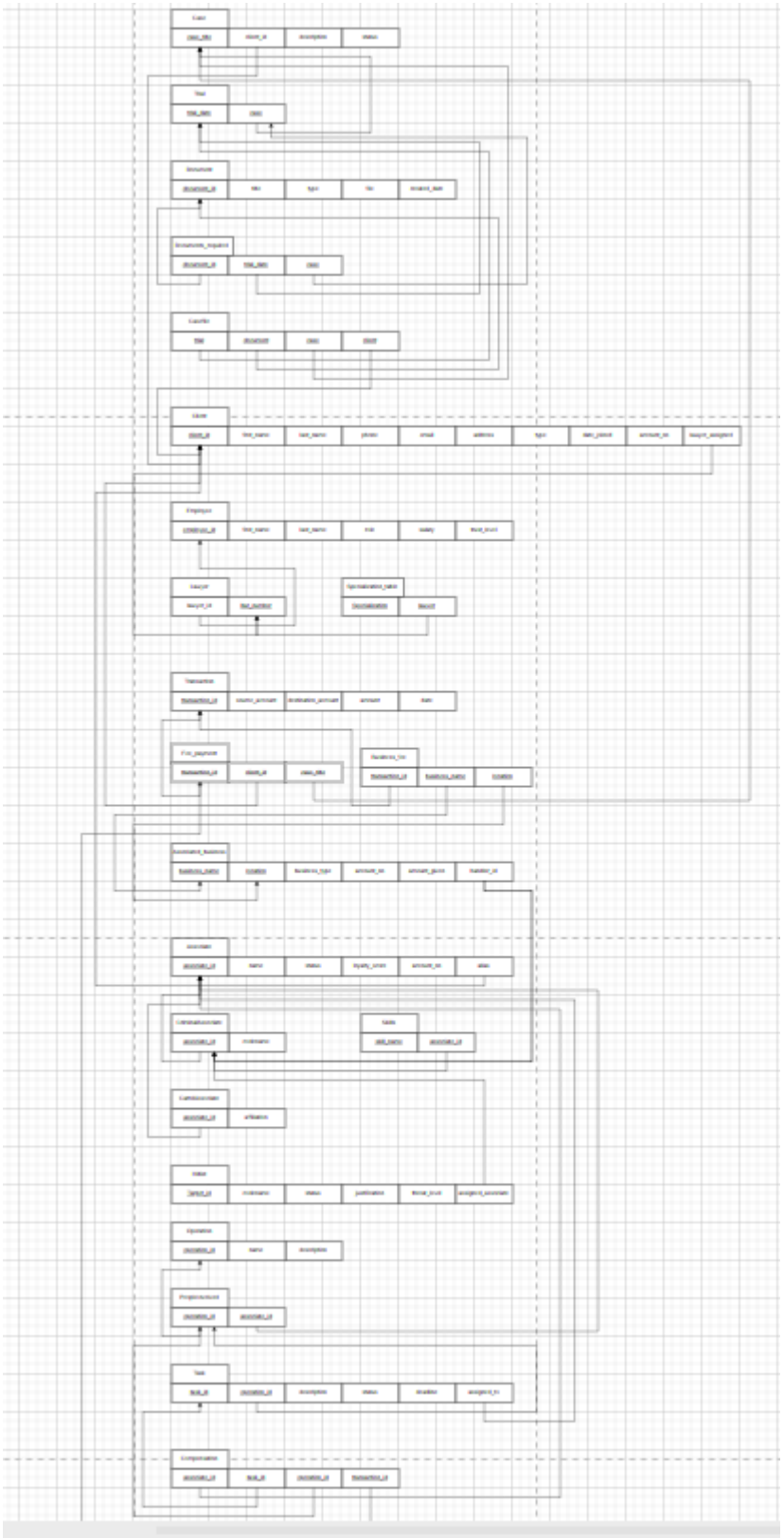
Binary 1:1 and 1:N Relationship Types: Most of them were converted to foreign keys. Sometimes we have to create a new relation that includes the primary keys of both

participating entities as foreign keys. The primary key would be a combination of both these foreign keys. Eg: Fee_payment and Business_fee were made as different tables.

Binary M:N Relationship Types: A new relation was made that includes the primary keys of both participating entities as foreign keys. The primary key would be a combination of both these foreign keys.

N-ary Relationship Types(N > 2): Create a new relation to represent the relationship. Include the primary keys of all participating entities as foreign keys. Add any simple attributes from the relationship. The primary key of this new relation is typically the combination of all the included foreign keys.

https://drive.google.com/file/d/1TNHkCSL7TAzIVVtXbIEbVnjB3yld8KAh/view?usp=drive_link



1NF:

Here each attribute can contain only atomic values. Since the procedure to convert ER diagram to relational model makes each attribute atomic, the relational model is already in 1NF.

2NF:

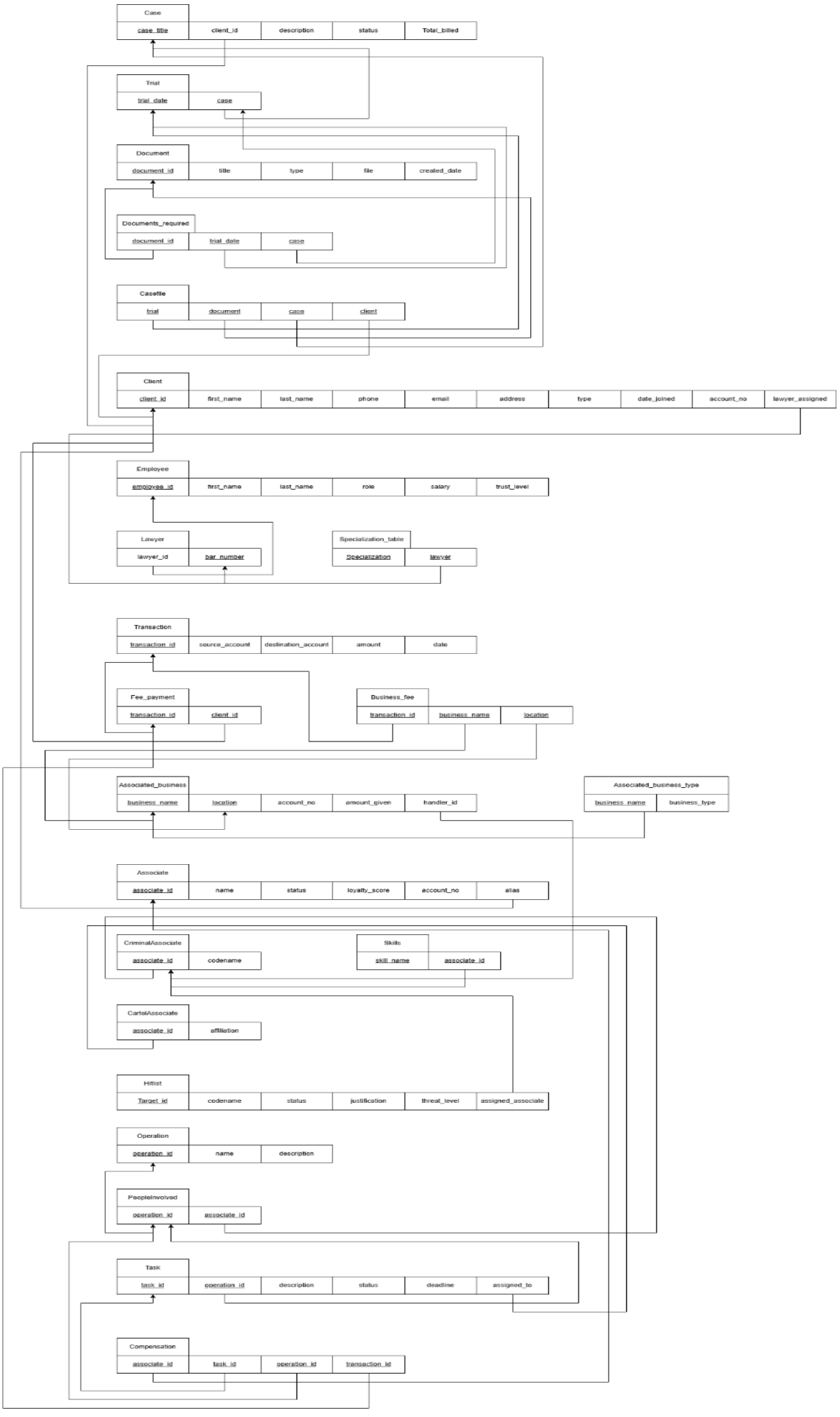
A relation schema R is in second normal form (2NF) if every non prime attribute A in R is not partially dependent on any key of R.

To convert from 1nf to 2nf, make new relation tables for FD's from a subset of a candidate key to non prime attributes, with the attributes it is dependent on as primary key. The elements on the right hand part of the dependency are removed from the original table. The elements on the left hand part of the dependency in the original table are foreign keys to the new table.

In Associated_business -> business_type is an FD. Hence it is separated into another relation.

Note account_no can be different for different locations of the same business, so its functionally dependent on both.

https://drive.google.com/file/d/1BqtjNzsHgosGPaZYkkhQ_GzaqhqukyXT/view?usp=drive_link


Case
case_title
client_id
description
status
Total_billed
Trial
trial_date
case
Document
document_id
title
type
file
created_date
Documents_required
document_id
trial_date
case
Casefile
trial
document
case
client
Client
client_id
first_name
last_name
phone
email
address
type
date_joined
account_no
lawyer_assigned
Employee
employee_id
first_name
last_name
role
salary
trust_level
Lawyer
lawyer_id
bar_number
Specialization_table
Specialization
lawyer
Transaction
transaction_id
source_account
destination_account
amount
date
Fee_payment
transaction_id
client_id
Business_fee
transaction_id
business_name
location
Associated_business
business_name
location
account_no
amount_given
handler_id
Associated_business_type
business_name
business_type
Associate
associate_id
name
status
loyalty_score
account_no
alias
CriminalAssociate
associate_id
codename
Skills
skill_name
associate_id
CartelAssociate
associate_id
affiliation
Hitlist
Target_id
codename
status
justification
threat_level
assigned_associate
Operation
operation_id
name
description
PeopleInvolved
operation_id
associate_id
Task
task_id
operation_id
description
status
deadline
assigned_to
Compensation
associate_id
task_id
operation_id
transaction_id

3NF:

A relation schema R is in third normal form (3NF) if, whenever a nontrivial functional dependency X → A holds in R, either (a) X is a superkey of R, or (b) A is a prime attribute of R.

On finding a functional dependency from a non-key attribute to non-prime attribute, partition the table to create a new table with the elements of the dependency, having the left hand part of the dependency as the primary key. The elements on the right hand part of the dependency are removed from the original table. The elements on the left hand part of the dependency in the original table are foreign keys to the new table.

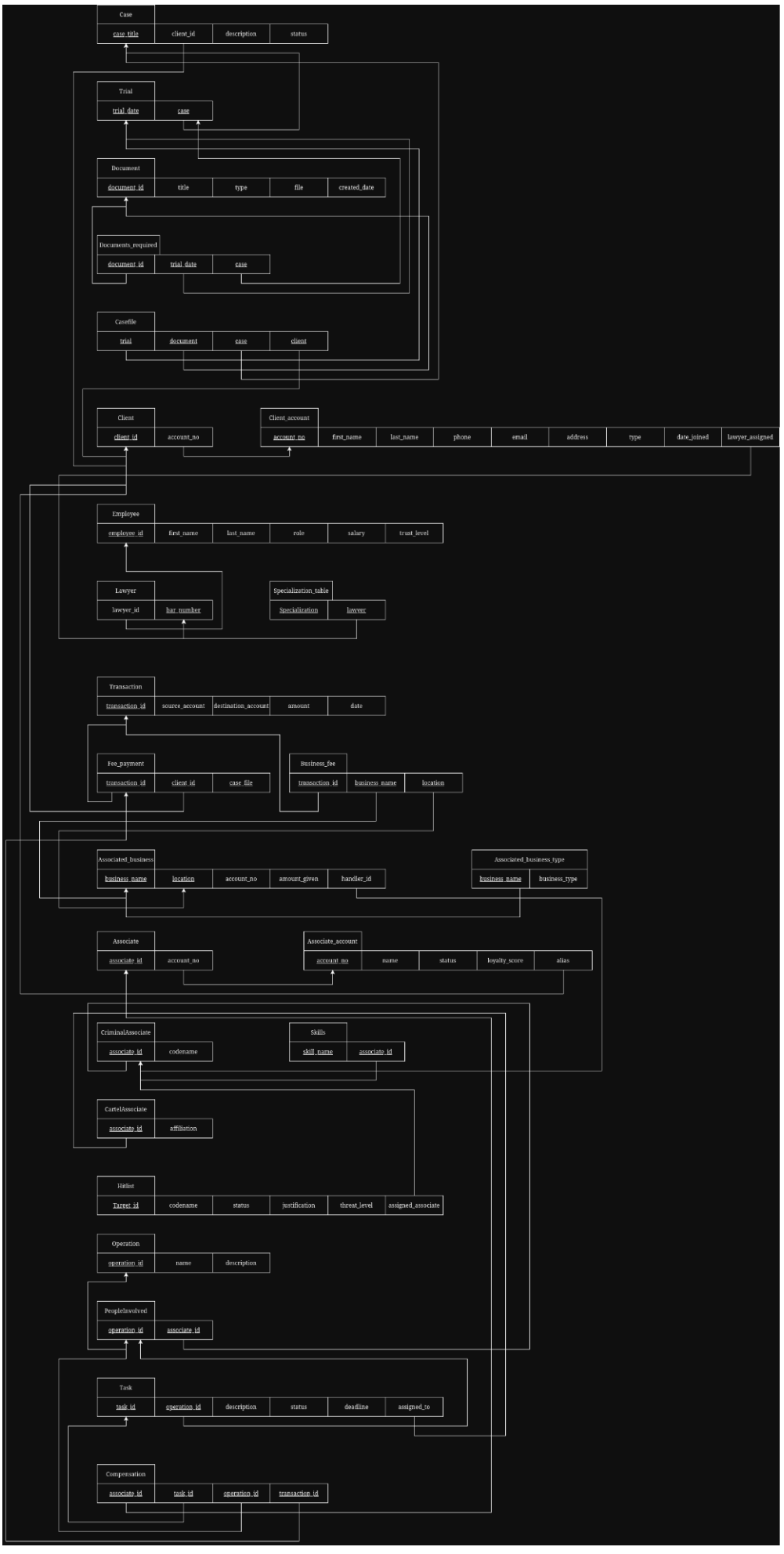
In the client table, attributes ‘phone number’, ‘email’ etc. may not be unique according to our DB. For example, people in a family may use common ones. Hence, it won’t violate 3NF FD constraints.

In the associate_business table, multiple locations can have the same account number used for transactions/business. So, the account number attribute cannot be used to uniquely identify a tuple. Hence, it won’t violate 3NF FD constraints.

One change we made is that we partition the associate table into 2 tables, as account id can also uniquely identify a tuple, so 3NF violation will occur as:
associate_id -> account_id -> non-key attributes (transitivity causes 3NF violation)
The new table we created (associate_account table) contains data related to the bank details (key being account number). This can be used to query account name, status, alias etc (attributes related to associate table).

Another change we made is that we partition the client table into 2 tables, as account id can also uniquely identify a tuple, so 3NF violation will occur as:
Client_id -> account_id -> non-key attributes (transitivity causes 3NF violation)
The table is split accordingly to remove the 3NF violation.

https://drive.google.com/file/d/1LQMw0ul5t-zsEsB8OKbna7wQqIFfqgid/view?usp=drive_link


Case
case_title
client_id
description
status
Trial
trial_date
case
Document
document_id
title
type
file
created_date
Documents_required
document_id
trial_date
case
Casefile
trial
document
case
client
Client
client_id
account_no
Client_account
account_no
first_name
last_name
phone
email
address
type
date_joined
lawyer_assigned
Employee
employee_id
first_name
last_name
role
salary
trust_level
Lawyer
lawyer_id
bar_number
Specialization_table
Specialization
lawyer
Transaction
transaction_id
source_account
destination_account
amount
date
Fee_payment
transaction_id
client_id
case_file
Business_fee
transaction_id
business_name
location
Associated_business
business_name
location
account_no
amount_given
handler_id
Associated_business_type
business_name
business_type
Associate
associate_id
account_no
Associate_account
account_no
name
status
loyalty_score
alias
CriminalAssociate
associate_id
codename
Skills
skill_name
associate_id
CartelAssociate
associate_id
affiliation
Hitlist
Target_id
codename
status
justification
threat_level
assigned_associate
Operation
operation_id
name
description
PeopleInvolved
operation_id
associate_id
Task
task_id
operation_id
description
status
deadline
assigned_to
Compensation
associate_id
task_id
operation_id
transaction_id